

Path Tracking & Control of Autonomous Vehicles

Using the Kinematic Bicycle Model

Sujal Singh — Personal Project

Github link-<https://github.com/SujalSingh555/autonomous-vehicle-path-tracking.git>

1. Objective

The **objective** of this project was to develop and compare different path-tracking strategies for autonomous vehicle navigation. **Pure Pursuit, Stanley, and Linear Quadratic Regulator (LQR) controllers** were implemented and evaluated on a **kinematic bicycle model** across multiple reference trajectories. Their performance was compared using **tracking accuracy, control behavior, and robustness to sensor noise**.

I chose a kinematic bicycle model because the primary goal of the project was to study and compare path-tracking algorithms rather than vehicle dynamics. Since the model is valid at moderate speeds where **lateral tyre slip and sideways friction forces** can be neglected, it provides a simple and computationally efficient platform for controller development. This allowed me to focus on controller design, tuning, and performance evaluation before moving to more complex dynamic vehicle models.

2. Vehicle Model

Kinematic Bicycle Model

The vehicle is modelled as a two-wheeled bicycle system. The model assumes no tyre slip and captures steering-induced motion using three coupled differential equations.

$$\begin{aligned}x_{\dot{}} &= v \cdot \cos(\psi) \\y_{\dot{}} &= v \cdot \sin(\psi) \\\psi_{\dot{}} &= (v / L) \cdot \tan(\delta)\end{aligned}$$

States and Inputs

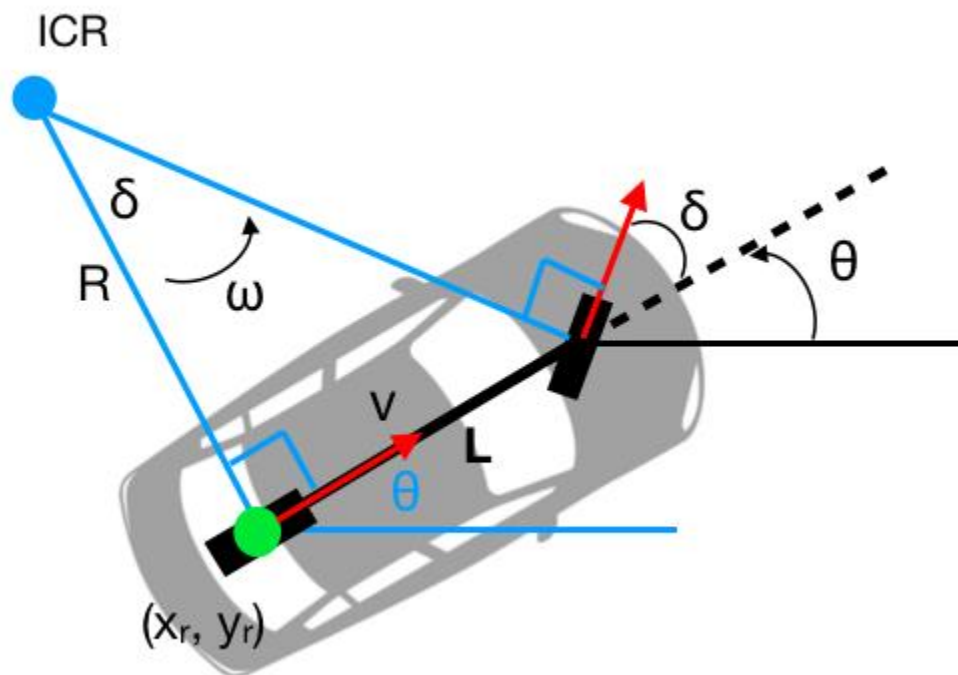
States: (x, y) — position, ψ — heading angle, v — velocity

Inputs: δ — steering angle, a — longitudinal acceleration

Wheelbase: $L = 4 \text{ m}$

Model Assumptions

- No lateral tyre slip.
- Pure rolling motion at the wheel-ground contact points.
- Low to moderate operating speeds where tyre force dynamics can be neglected.
- Front and rear wheels rotate about a common Instantaneous Center of Rotation (ICR) during turning.
- Left and right wheels are combined into equivalent front and rear wheels, resulting in a two-wheel bicycle representation.



3. Path Tracking Controllers

3.1 Pure Pursuit

A geometric controller that steers toward a lookahead point on the reference path at a fixed distance L_d from the rear axle.

$$\delta = \arctan(2*L*\sin(\alpha)/L_d)$$

Tuning parameter: L_d — lookahead distance (larger = smoother, less accurate)

- Lookahead point found by intersecting a circle of radius L_d with the path
- Steering computed from the lateral offset of the lookahead point in the robot frame.

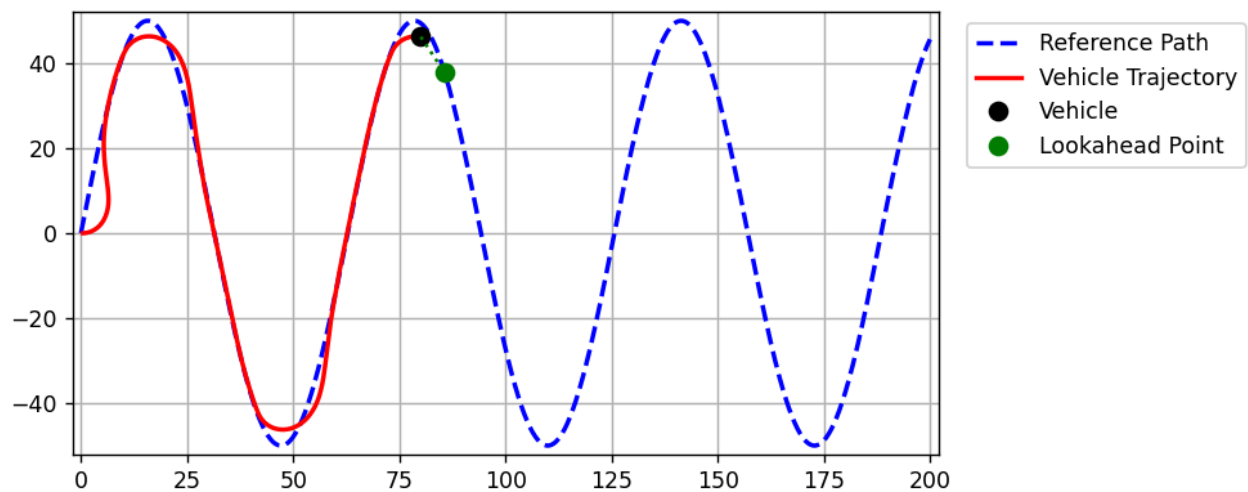


Figure 1: Pure Pursuit controller tracking a sinusoidal reference trajectory. The black marker indicates the current vehicle position, while the green marker represents the lookahead point used for steering computation.

3.2 Stanley

An error-based controller measuring both heading error and cross-track error at the front axle.

$$\delta = k_{\psi} \cdot \psi_e + \arctan(k \cdot cte / v)$$

psi_e: heading error — angle between vehicle heading and path tangent

cte: cross-track error — perpendicular distance from front axle to path

k: cross-track gain

k_psi: heading gain (extended Stanley — improves curve response)

- CTE measured and signed at the front axle via cross product
- arctan naturally saturates steering for large errors
- Speed v in denominator gives automatic gain scheduling

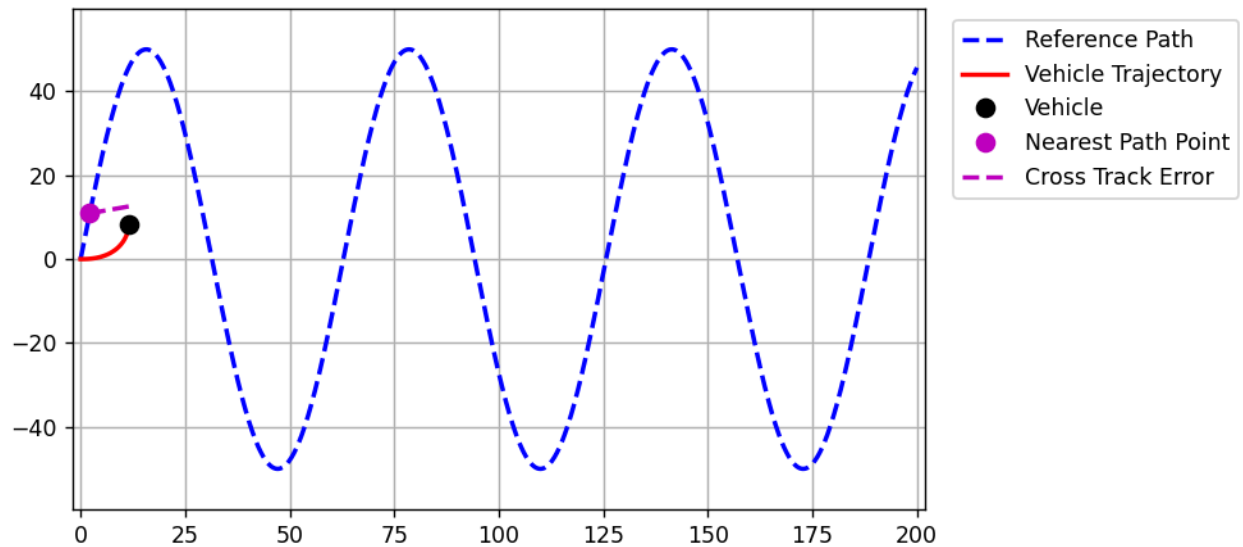


Figure 2: Stanley controller geometry. The magenta marker indicates the nearest point on the reference path, while the dashed line represents the cross-track error used for steering correction. Steering commands are generated using both cross-track error and heading error.

3.3 LQR (Linear Quadratic Regulator)

An optimal state-space controller that minimises a quadratic cost function penalising tracking error and control effort simultaneously.

$$J = \int (x^T Q x + u^T R u) dt$$

State vector: $x = [e, e_dot, psie, psie_dot, v_e]$

Control input: $u = [\delta_dot, a]$ (steering rate + acceleration)

The optimal control law $u^* = -Kx$ is found by solving the Algebraic Riccati Equation (ARE):

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

$$K = R^{-1} B^T P$$

System matrices (kinematic bicycle, linearised around v_ref):

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & v & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ v/L & 0 \\ 0 & 1 \end{bmatrix}$$

Q: `np.diagflat([10, 1, 10, 1, 4])` — state error weights

R: `np.diagflat([1, 1])` — control effort weights

- ARE cached and re-solved only when speed changes by > 0.5 m/s
- Curvature-adaptive target speed: $v_ref = v_max / (1 + 10 \cdot \kappa)$
- Input is δ (steering rate) for smoother wheel motion; integrated to get δ

4. Sensor Noise Modelling & State Estimation

Noise Model

Gaussian noise was added to position and heading measurements to simulate realistic sensor conditions:

$$\begin{aligned}x_{meas} &= x + N(0, \sigma_{xy}) \\y_{meas} &= y + N(0, \sigma_{xy}) \\psi_{meas} &= \psi + N(0, \sigma_{\psi})\end{aligned}$$

Position noise σ_{xy} : 0.2 m=20 cm

Heading noise σ_{ψ} : 0.03 rad=1.17 radian

Kalman Filter

A Kalman Filter was implemented to estimate the vehicle state ((x, y, psi)) from noisy position and heading measurements. The filter combines a model-based prediction of vehicle motion with noisy sensor observations to obtain a more accurate estimate of the vehicle state.

- Prediction step uses the kinematic bicycle model to propagate the state estimate forward in time.
- Update step incorporates noisy position and heading measurements to correct the predicted state.
- The estimated state is supplied to the path-tracking controller instead of raw sensor measurements.
- State estimation reduced measurement fluctuations and improved tracking performance under noisy conditions.

For the sinusoidal trajectory, the use of Kalman filtering reduced the RMS tracking error of the LQR controller from approximately 0.9 m to 0.7 m.

5. Experimental Setup

Test Trajectories

- Straight Path
- Circular Path
- Lane Change Path
- Sinusoidal Path

Simulation Parameters

Simulation frequency: 20 Hz ($dt = 0.05$ s)

Vehicle speed (PP / Stanley): 10 m/s (constant)

Max speed (LQR): 10 m/s (adaptive)

Wheelbase: $L = 4$ m

Max steering angle: $\pm 35^\circ$

Max iterations: 5000 steps per run

6. Results

6.1 Trajectory Comparison

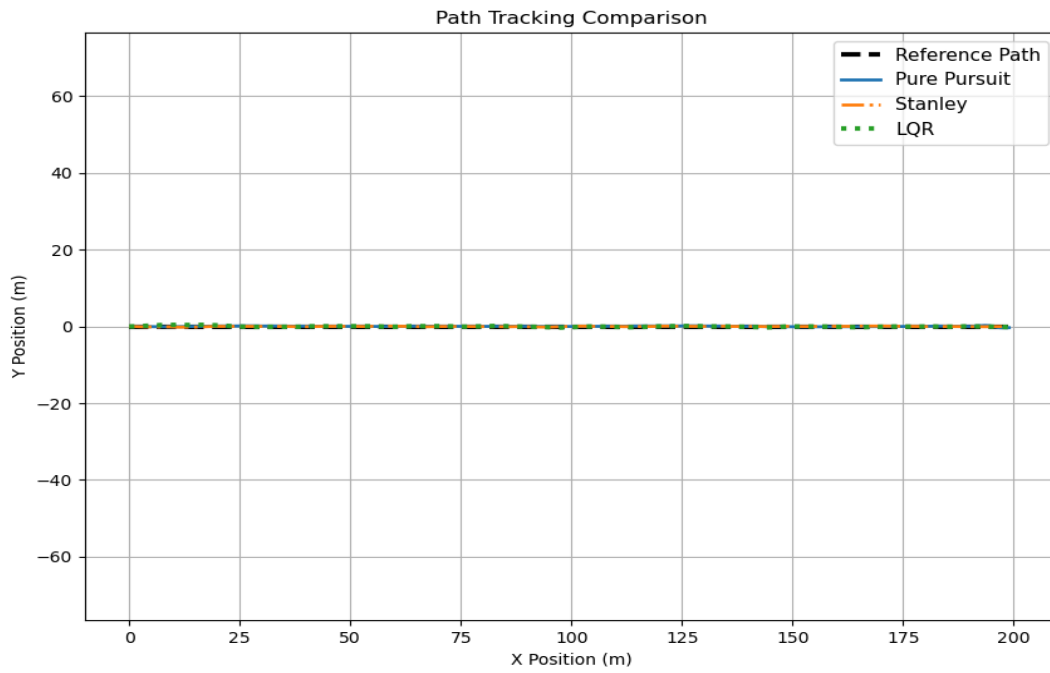


Figure 3: Straight-line reference trajectory defined by $y=0$, $x \in [0, 200]$. This path represents a zero-curvature scenario and serves as a baseline benchmark for evaluating controller tracking accuracy and steady-state performance.

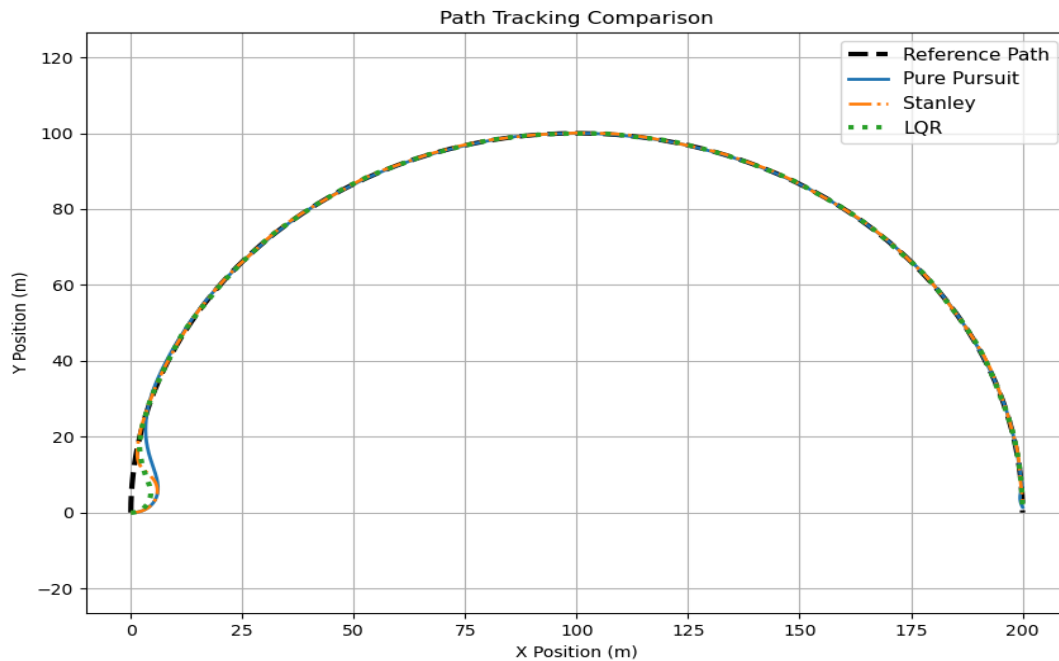


Figure 4: Semi-circular reference trajectory generated using $x=R\sin(\theta)$, $y=R\cos(\theta)$, where $R=200$ m and $\theta \in [0, \pi]$. This trajectory evaluates controller performance under continuous curvature and sustained steering action.

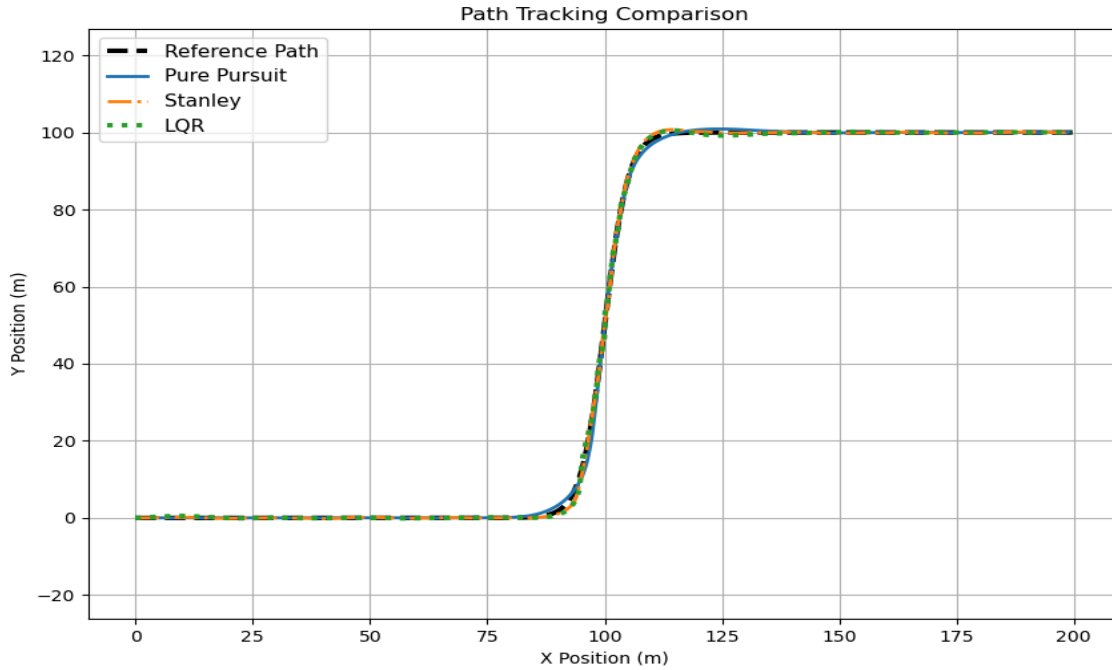


Figure 5: Lane-change trajectory generated using $y=W\tanh(x-L/25)-W\tanh(-L/25)$, where $W=50$ m and $L=200$ m. The smooth lateral transition produced by the hyperbolic tangent function is used to evaluate controller response during rapid path changes.

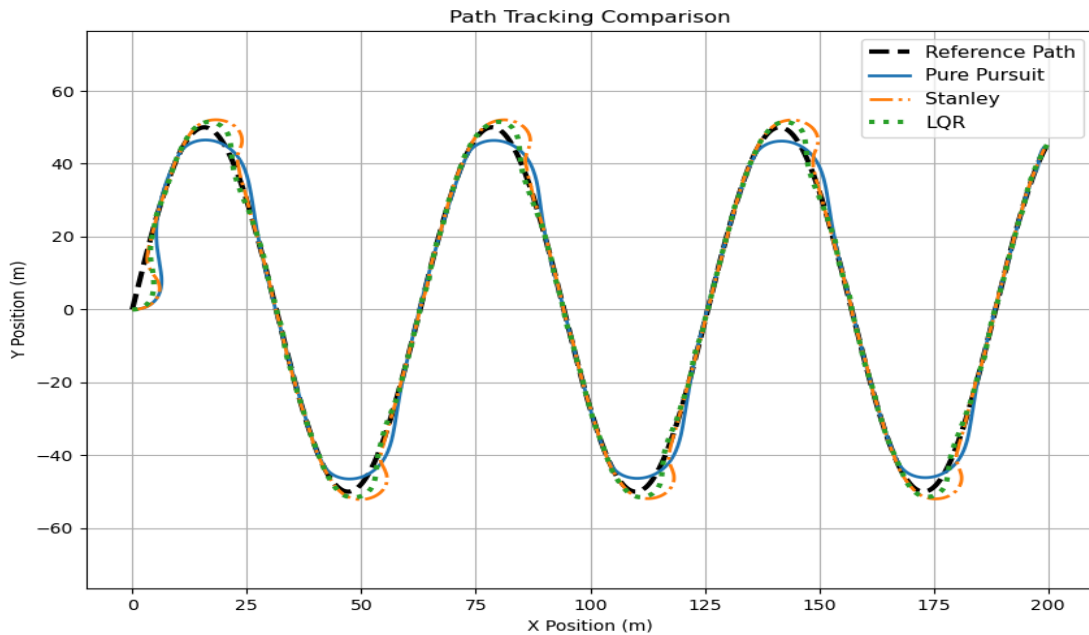


Figure 6: Sinusoidal reference trajectory defined by $y=A\sin(fx)$, where $A=50$ and $f=0.1$. The continuously varying curvature provides a challenging benchmark for assessing tracking accuracy, responsiveness, and controller stability.

6.2 Error Comparison

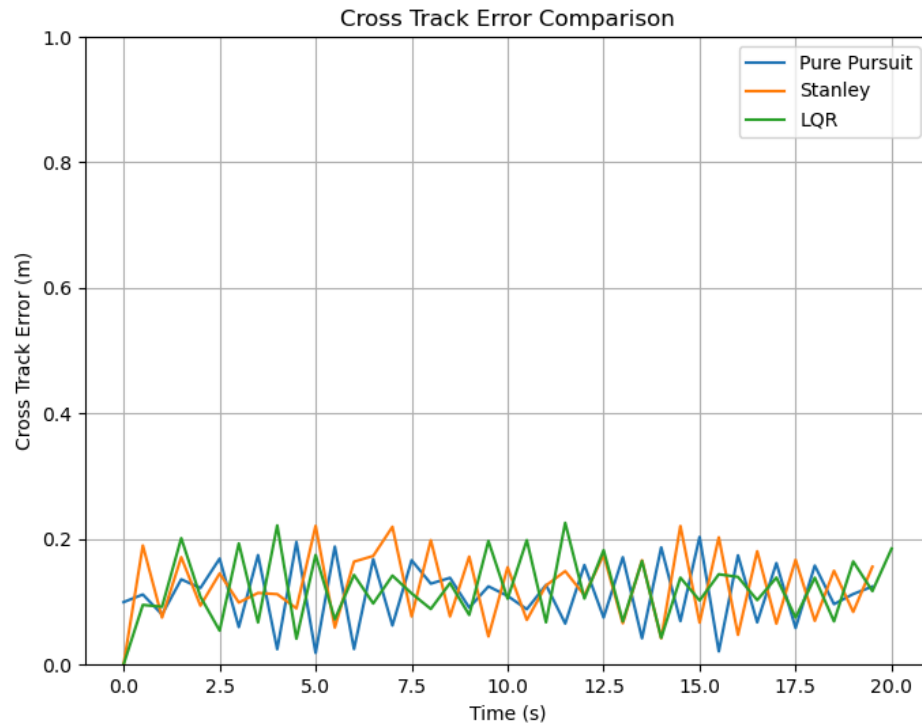


Figure 7: Cross-track error comparison for the **straight-line** trajectory

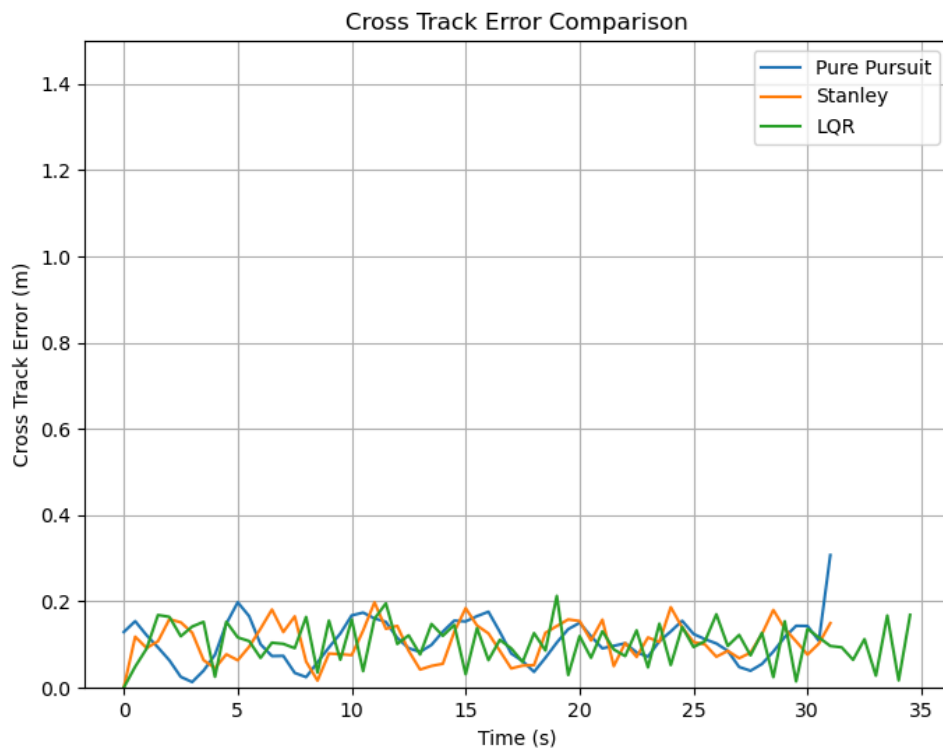


Figure 8: Cross-track error comparison for the **semi-circular** trajectory.

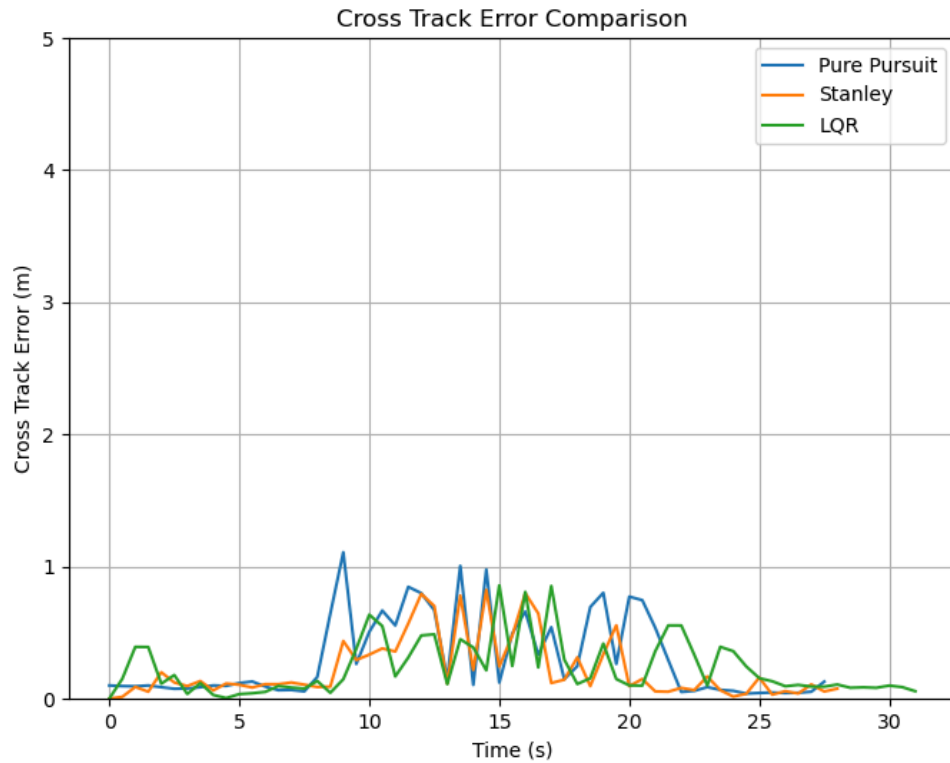


Figure 9: Cross-track error comparison for the **lane-change** trajectory.

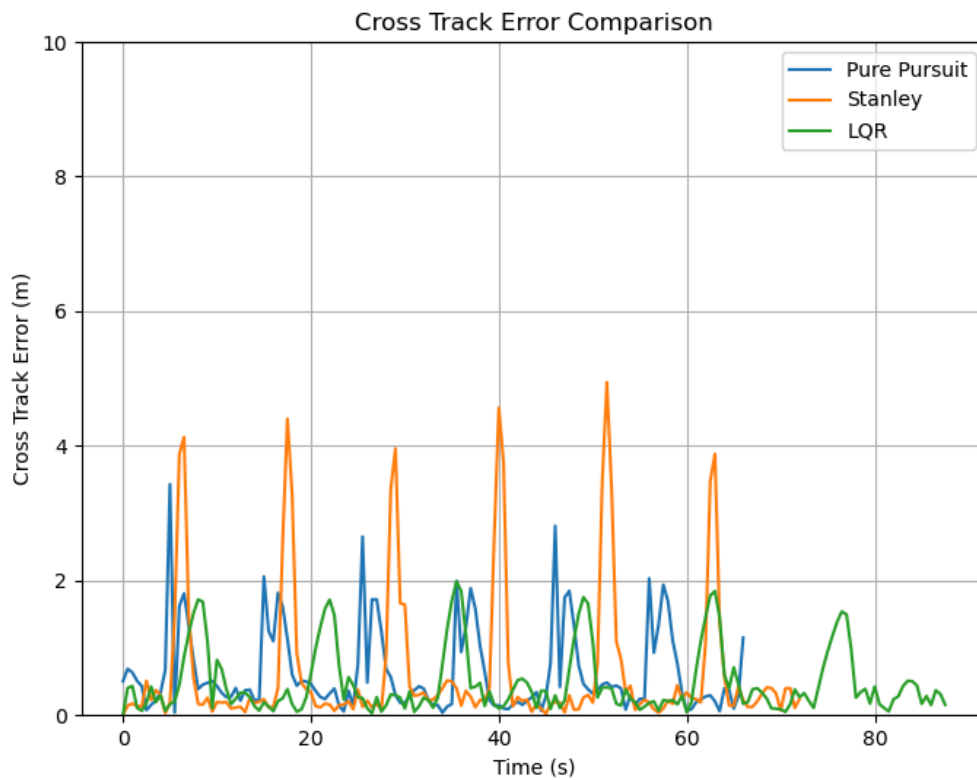


Figure 10: Cross-track error comparison for the **sinusoidal** trajectory.

6.3 Performance Metrics

Straight Line Path

Controller	RMS Error (m)	Max Error (m)	Mean Error (m)
Pure Pursuit	0.143	0.595	0.125
Stanley	0.134	0.277	0.122
LQR	0.142	0.317	0.130

Semi Circular Path

Controller	RMS Error (m)	Max Error (m)	Mean Error (m)
Pure Pursuit	0.111	0.443	0.098
Stanley	0.113	0.214	0.104
LQR	0.124	0.367	0.109

Lane Change Path

Controller	RMS Error (m)	Max Error (m)	Mean Error (m)
Pure Pursuit	0.435	1.178	0.321
Stanley	0.304	0.968	0.212
LQR	0.300	0.992	0.264

Sine Path

Controller	RMS Error (m)	Max Error (m)	Mean Error (m)
Pure Pursuit	0.881	3.35	0.617
Stanley	1.102	4.102	0.635
LQR	0.671	1.813	0.486

7. Observations

Pure Pursuit

Pure Pursuit was the easiest controller to implement and tune. It performed well on straight and circular trajectories and required only the lookahead distance as the main tuning parameter. However, on trajectories with rapidly changing curvature, such as the lane-change and sinusoidal paths, the controller **showed larger tracking errors** because steering decisions are based on a future lookahead point rather than the instantaneous tracking error. .

Stanley

Stanley generally performed better than Pure Pursuit on aggressive maneuvers because it directly corrects both heading error and cross-track error. During tuning, I observed that the controller was quite sensitive to the heading gain (**k_psi**). Increasing the gain beyond a certain value caused oscillatory steering behaviour, while lower values resulted in slower error correction.

LQR

LQR achieved the best performance on the more challenging trajectories, particularly the sinusoidal path, where it produced the **lowest RMS and maximum tracking errors**. However, it was also the most difficult controller to design and tune. Controller performance depended heavily on the choice of states and the selection of the **Q and R weighting matrices**. During development, **larger state penalties occasionally caused the Riccati solver to fail or produced unstable controller behaviour**, requiring careful tuning of the weighting matrices.

Noise Robustness

Gaussian noise was added to the position and heading measurements to simulate sensor uncertainty. A Kalman Filter was implemented to estimate the vehicle state from these noisy measurements before passing it to the controllers. While the improvement for Pure Pursuit and Stanley was relatively small, the LQR controller benefited noticeably from state estimation. For the sinusoidal trajectory, the RMS tracking error reduced from **approximately 0.9 m to 0.7 m, corresponding to an improvement of about 22%**.

All controllers achieved similar performance on straight and circular trajectories. Performance differences became more apparent on lane-change and sinusoidal trajectories, where curvature variations and rapid heading changes increased tracking difficulty. .

8. Conclusion

This project explored the implementation and comparison of three classical path-tracking controllers Pure Pursuit, Stanley, and LQR using a kinematic bicycle model. Through testing on different trajectories, I observed that all three controllers were able to track simple paths such as straight lines and circular curves with relatively small errors. However, the differences between the controllers became much more apparent on lane-change and sinusoidal trajectories, where curvature changed more rapidly.

Among the three controllers, LQR achieved the best overall performance on the more challenging paths, although it also required the most effort in terms of modelling, state selection, linearisation, and tuning of the Q and R matrices. I spent a significant amount of time understanding how controller parameters affected stability and tracking performance, particularly when the Riccati solver became sensitive to some specific weighting choice.

The project also provided practical experience with sensor noise modelling and state estimation. Incorporating a Kalman Filter produced smoother state estimates and improved the performance of the LQR controller under noisy conditions, reducing the RMS tracking error on the sinusoidal trajectory from approximately 0.9 m to 0.7 m approximately reducing cross track error by 22 percent.

Overall, this project served as an introduction to vehicle modelling, path-tracking control, optimal control, and state estimation, while also highlighting several areas for future exploration such as dynamic vehicle models, Model Predictive Control (MPC), and ROS2-based simulations